Отчёт по индивидуальному заданию № 2

Архитектура компьютера и операционные системы

Вариант 9. Магазин с несколькими отделами

ФИО: Жиляков Роман Евгеньевич

Группа: БПИ252

1. Условие задания

В магазине работают три отдела, в каждом находится один продавец. Покупатель выбирает один или несколько различных отделов и порядок их посещения.

Если продавец занят, покупатель становится в очередь соответствующего отдела. После завершения обслуживания покупатель переходит в следующий отдел своего маршрута. Выполнив весь план покупок, он покидает магазин.

Необходимо реализовать многопоточную имитацию работы магазина на языке C для Linux. Покупатели и продавцы должны быть представлены отдельными потоками, взаимодействующими между собой асинхронно с использованием мьютексов и семафоров.

Программа должна отображать действия покупателей и продавцов, сохранять журнал работы и поддерживать обычное завершение и прерывание.

2. Описание модели

Программа выполняется в одном процессе с использованием библиотеки pthread. В отличие от ИДЗ № 1, действия участников выполняются отдельными потоками, взаимодействие которых организовано с помощью синхропримитивов.

В модели предусмотрены три отдела и не более 100 покупателей. Каждый покупатель посещает от одного до трёх различных отделов. Маршрут задаётся во входном файле.

Для каждого покупателя создаётся отдельный поток buyer_thread, который отвечает за вход, переходы между отделами и завершение покупок. Для каждого отдела создаётся поток продавца seller_thread, который обслуживает покупателей своей очереди.

Главный поток управляет модельным временем, а дополнительный поток signal_thread обрабатывает сигналы завершения.

Таким образом, при N покупателях создаётся N + 5 потоков, включая главный.

Модельное время измеряется целыми тактами. Первый покупатель появляется в момент 0, каждый следующий — через заданный интервал:

Время появления покупателя i = (i − 1) × INTERVAL

Продолжительность обслуживания выбирается в заданном для отдела диапазоне. Если минимальное и максимальное значения совпадают, длительность обслуживания постоянна.

Для выбора случайной длительности используется rand_r с отдельным состоянием генератора у каждого продавца.

Очереди обслуживаются по правилу FIFO: первым обслуживается покупатель, который раньше встал в очередь. Для каждого отдела используется массив на 100 номеров покупателей. Этого достаточно, поскольку каждый покупатель посещает один отдел не более одного раза.

Покупатель находится в одном из шести состояний:

Состояние	Значение
NOT_ARRIVED	Ещё не пришёл
WAITING	Ожидает в очереди
SERVING	Обслуживается
READY	Завершил обслуживание и готов к переходу или выходу
LEFT	Выполнил маршрут и вышел
REJECTED	Не допущен после закрытия входа

При наступлении времени CLOSE вход закрывается. Покупатель, время появления которого совпадает с CLOSE, не допускается в магазин.

Уже вошедшие покупатели после закрытия продолжают обслуживание и полностью выполняют свои маршруты. Покупатели, которые не успели войти, отмечаются в журнале как не допущенные.

Если все покупатели вышли раньше закрытия, программа ожидает заданного времени закрытия.

Для хранения состояния используются структуры Buyer, Department и Shop.

Структура Buyer содержит маршрут, текущее положение, состояние покупателя, поток и семафор уведомлений.

Структура Department содержит параметры обслуживания, очередь FIFO, информацию о текущем покупателе, поток продавца и семафоры.

Структура Shop содержит общие параметры модели, текущее время, состояние всех участников, мьютекс, условную переменную и информацию о завершении программы.

3. Алгоритм работы

При запуске программа загружает исходные данные, инициализирует структуры и средства синхронизации, после чего создаёт потоки покупателей и продавцов.

Главный поток выполняет роль планировщика событий. В отличие от последовательной модели, он не перебирает все целые такты, а определяет ближайший момент, когда должно произойти очередное событие.

Алгоритм работы:

1. Считываются исходные параметры магазина и маршруты покупателей.
2. Инициализируются мьютекс, семафоры и условная переменная.
3. Создаются потоки продавцов, покупателей и обработки сигналов.

4. Главный поток определяет ближайшее событие: приход покупателя, окончание обслуживания или закрытие входа.
5. При наступлении события соответствующим потокам отправляются уведомления.
6. Потоки покупателей и продавцов обрабатывают полученные уведомления и изменяют состояние модели.
7. Главный поток ожидает обработки всех событий текущего модельного времени.
8. Проверяется возможность завершения или выбирается следующий момент времени.
9. После завершения моделирования выполняется остановка потоков, освобождение ресурсов и сохранение итоговой статистики.

Для взаимодействия потоков используются следующие механизмы.

Мьютекс pthread_mutex_t

Общий мьютекс mu защищает состояние магазина, очереди, статистику и журнал событий. Он предотвращает одновременное изменение общих данных несколькими потоками.

Семафоры sem_t

Для каждого покупателя используется семафор gate, через который передаются разрешение на вход и уведомление об окончании обслуживания.

Для каждого продавца предусмотрены два семафора:

- work — уведомление о появлении работы в очереди;
- finish — разрешение завершить обслуживание при наступлении соответствующего модельного времени.

Ожидание организовано через sem_wait, отправка уведомлений — через sem_post.

Условная переменная pthread_cond_t

Условная переменная changed используется совместно со счётчиком pending. Счётчик отражает количество отправленных, но ещё не обработанных уведомлений.

Главный поток ожидает, пока значение pending не станет равным нулю. Это предотвращает переход к следующему моменту модельного времени до завершения обработки текущих событий.

При одновременных событиях порядок выполнения отдельных потоков определяется планировщиком ОС. Поэтому порядок входа покупателей, прибывших одновременно, может различаться между запусками.

При этом FIFO сохраняется: покупатели обслуживаются именно в порядке фактической постановки в очередь.

Программа поддерживает следующие правила:

- продавец обслуживает только одного покупателя одновременно;
- покупатель находится только в одном месте;
- отделы в маршруте не повторяются;
- следующий отдел посещается после завершения обслуживания в предыдущем;
- очередь каждого отдела обслуживается в порядке FIFO;

- обычный выход покупателя происходит после выполнения всего маршрута;
- после закрытия новые покупатели не допускаются;
- при штатном завершении все вошедшие покупатели завершают покупки.

Обычный запуск завершается, поскольку время закрытия конечно, количество покупателей ограничено, а каждый маршрут содержит не более трёх обслуживаний конечной длительности.

4. Состав проекта

Основные файлы находятся в папке hw2.

Файл или каталог	Назначение
main.c	Исходный код многопоточной программы
Makefile	Сборка, запуск тестов и удаление исполняемых файлов
tests/*.txt	Входные данные для проверок
tests/run.sh	Скрипт автоматического тестирования
tests/validate.py	Проверка событий, очередей FIFO и инвариантов
report.docx	Редактируемый отчёт
report.pdf	Итоговый отчёт
.gitignore	Исключение служебных и автоматически созданных файлов

При сборке создаётся исполняемый файл shop. Журналы запусков сохраняются в каталоге logs.

Исполняемый файл и автоматически полученные журналы не входят в комплект исходных файлов.

5. Интерфейс и формат входных данных

Запуск программы:

./shop INPUT_FILE LOG_FILE DELAY_MS SEED

Аргументы:

Аргумент	Назначение
INPUT_FILE	Путь к входному файлу
LOG_FILE	Путь к журналу
DELAY_MS	Реальная задержка моделирования в миллисекундах на один модельный такт
SEED	Начальное значение генератора случайных чисел

Входной файл и журнал должны быть обычными файлами. Каталог для журнала должен существовать.

Реальная задержка не изменяет модельное время. Для автоматических тестов используется задержка 0.

Одинаковый SEED задаёт одинаковые начальные состояния генераторов случайных чисел. Однако из-за различий в порядке выполнения потоков полный журнал событий не обязательно будет одинаковым при повторных запусках.

Допустимые значения:

Параметр	Диапазон
Количество покупателей N	От 0 до 100
Интервал появления INTERVAL	От 0 до 100000 тактов
Время закрытия CLOSE	От 0 до 100000 тактов
Время обслуживания MIN, MAX	От 1 до 10000 тактов, MIN ≤ MAX
Длина маршрута K	От 1 до 3
Номера отделов	От 1 до 3, без повторений
Задержка DELAY_MS	От 0 до 1000 мс
SEED	От 0 до INT_MAX
Размер входного файла	Не более 65534 байт

Формат входного файла:

```
N INTERVAL CLOSE
MIN1 MAX1
MIN2 MAX2
MIN3 MAX3
K D1 ... DK
...
```

После параметров отделов указываются N маршрутов покупателей.

Пример:

```
3 0 5
2 2
2 2
2 2
2 1 2
1 1
2 3 1
```

В этом примере:

- все три покупателя появляются в момент 0;
- вход закрывается в момент 5;
- обслуживание в каждом отделе занимает два такта;
- покупатель 1 посещает отделы 1 и 2;
- покупатель 2 посещает отдел 1;
- покупатель 3 посещает отделы 3 и 1.

Программа отклоняет неверные числа, значения вне диапазона, повторные отделы, неполные и лишние данные, переполнение и нулевые байты во входном файле.

6. Сборка и запуск

Из корня проекта:

```
make -C hw2
```

make -C hw2 test

Ручной запуск:

cd hw2

mkdir -p logs

./shop tests/normal.txt logs/manual.log 200 1

Вывод справки:

./shop --help

Удаление исполняемых файлов:

make clean

Дополнительная сборка с AddressSanitizer и UndefinedBehaviorSanitizer:

make sanitize

Для сборки необходим GCC с поддержкой POSIX Threads. Для тестирования используются Bash, Python 3 и стандартные утилиты GNU/Linux, включая timeout.

Программа собирается в стандарте C11 с параметрами:

-D_POSIX_C_SOURCE=200809L -Wall -Wextra -Wpedantic -Wshadow -Wconversion -O2 -pthread

Параметр -pthread обеспечивает компиляцию и компоновку программы с поддержкой многопоточности.

7. Ввод-вывод и журнал

Входной файл читается через файловый дескриптор с помощью open, read и close. Чтение учитывает частичный результат, конец файла и прерывание системного вызова.

Числа разбираются с помощью strtol, после чего проверяются диапазон и границы токена.

События записываются в стандартный вывод и журнал через write. Обрабатываются частичная запись и ошибка EINTR. Диагностические сообщения направляются в стандартный поток ошибок.

В многопоточной программе запись событий дополнительно защищена общим мьютексом. Это необходимо, чтобы сообщения разных потоков не перемешивались и журнал сохранял целостную последовательность событий.

Перед очисткой журнала программа проверяет, что он является обычным файлом и не совпадает с входным файлом. Проверка учитывает символические и жёсткие ссылки. Это защищает входные данные от перезаписи.

В журнале отражаются:

- вход покупателя и его маршрут;

- переход между отделами;
- постановка в очередь;
- начало и завершение обслуживания;
- выход покупателя;
- закрытие входа и отказ во входе;
- причина завершения;
- итоговая статистика и незавершённые маршруты.

Каждое событие содержит модельное время в формате [t=...].

При штатной работе одинаковые строки событий записываются в терминал и журнал. При прерывании вывод в терминал может быть остановлен, но итоговое состояние сохраняется в журнале.

8. Завершение программы

При обычном завершении вход закрыт, все вошедшие покупатели вышли, очереди пусты, продавцы свободны.

Для прерывания предусмотрены сигналы SIGINT и SIGTERM. При ручном запуске SIGINT можно отправить сочетанием клавиш Ctrl+C.

В многопоточной реализации сигналы SIGINT и SIGTERM блокируются для рабочих потоков. Отдельный поток ожидает их с помощью sigwait.

После получения сигнала программа устанавливает признак остановки и разблокирует потоки, ожидающие на семафорах.

Затем главный поток дожидается завершения созданных потоков с помощью pthread_join. После этого формируется итоговая статистика, уничтожаются семафоры, мьютекс и условная переменная, закрывается журнал.

Для стандартного вывода в канал используется неблокирующий режим и ожидание через poll. Это позволяет обработать прерывание, даже если канал вывода заполнен.

При прерывании покупатели, оставшиеся внутри, не считаются успешно завершившими маршрут. В журнале сохраняются их фактические состояния.

Коды завершения:

Код завершения	**Значение**
0	Обычное завершение или успешный вывод справки
1	Ошибка аргументов, данных, файлов, вывода или выполнения
130	Прерывание сигналом SIGINT
143	Прерывание сигналом SIGTERM

Проверяемые балансы покупателей:

Вошли = вышли + внутри

Всего = вошли + не допущены + ещё не пришли

После обычного завершения число покупателей внутри и число ещё не пришедших равны нулю.

9. Тестирование и результаты

Проверка выполнена в среде Linux. Программа успешно собрана компиляторами GCC и Clang без предупреждений с указанными параметрами сборки.

В проверенной версии скрипта успешно выполнены **26 групп тестов**. Некоторые группы включают несколько запусков: например, проверку 20 значений SEED.

Сохранены все 20 исходных файлов тестовых данных ИДЗ № 1.

Скрипт использует validate.py для анализа журнала и проверки порядка FIFO, маршрутов, занятости продавцов, времени окончания обслуживания, однократности входа и выхода, а также итоговой статистики.

Результаты основных сценариев:

Сценарий	Полученный результат
normal	Завершение в момент 6; вошли и вышли 3 покупателя; покупки по отделам: 3, 1, 1
closing	Завершение в момент 8; вошли и вышли 2 покупателя; один не допущен
empty	Завершение в момент 5; покупателей нет
one	Единственный покупатель завершил маршрут и вышел
route	Маршрут 3 → 1 → 2 завершён в момент 6
fifo	Четыре покупателя обслужены в порядке фактической постановки в очередь; завершение в момент 8
parallel	В трёх отделах обслуживание началось в момент 0; завершение в момент 3
boundary	Покупатель, появившийся в момент закрытия, не допущен
closed	При закрытии в момент 0 никто не вошёл; зарегистрированы два отказа
100 покупателей	Все вышли; выполнено 300 покупок
random, 20 значений SEED	Все восемь покупателей вышли; проверены маршруты, FIFO и итоговая статистика
SIGINT, SIGTERM	Получены коды 130 и 143; состояния оставшихся покупателей сохранены

В нагрузочном сценарии использовались 100 покупателей, каждый из которых посещал все три отдела. Проверка выполнена с тремя различными значениями SEED. Все 300 обслуживаний завершились.

Проверены ошибочные данные: повтор отдела, неверный номер отдела, перевёрнутый диапазон обслуживания, нулевая длительность, отрицательные значения, переполнение, повреждённый числовой токен, обрыв и лишние данные.

Также проверены отсутствующий входной файл, каталог вместо файла, неверные аргументы и невозможность открыть журнал.

Отдельные проверки подтвердили защиту входного файла от перезаписи через обычный путь, символическую и жёсткую ссылки.

Для SIGINT и SIGTERM проверены завершение программы, отсутствие зависания при ожидающих потоках и сохранение состояния покупателей.

Дополнительно сценарий random был проверен с AddressSanitizer и UndefinedBehaviorSanitizer. Ошибок памяти и неопределённого поведения в данном запуске не обнаружено.

Все автоматические тесты завершились успешно.

10. Примеры работы

Основной сценарий

Пример событий одного из запусков normal.txt:

[t=0] Начало: покупатель=3 отдел=3 окончание=2

[t=0] Начало: покупатель=2 отдел=1 окончание=2

[t=2] Конец: покупатель=3 отдел=3

[t=2] Конец: покупатель=2 отдел=1

[t=2] Выход: покупатель=2 план=выполнен

[t=2] Начало: покупатель=1 отдел=1 окончание=4

[t=4] Конец: покупатель=1 отдел=1

[t=4] Начало: покупатель=3 отдел=1 окончание=6

[t=4] Начало: покупатель=1 отдел=2 окончание=6

[t=5] Вход закрыт

[t=6] Выход: покупатель=1 план=выполнен

[t=6] Выход: покупатель=3 план=выполнен

[t=6] Итог: причина=завершено

[t=6] Покупатели: всего=3 вошли=3 вышли=3 внутри=0 не_допущены=0 ещё_не_пришли=0

[t=6] Отдел: номер=1 покупок=3 очередь=0 обслуживается=0

[t=6] Отдел: номер=2 покупок=1 очередь=0 обслуживается=0

[t=6] Отдел: номер=3 покупок=1 очередь=0 обслуживается=0

В приведённом запуске покупатели обслуживались в разных отделах одновременно. Все покупатели выполнили свои маршруты, после чего программа завершилась.

Обслуживание после закрытия

[t=3] Вход закрыт

[t=3] Отказ: покупатель=3 причина=вход_закрыт плановое_прибытие=4

[t=4] Конец: покупатель=1 отдел=1

[t=4] Начало: покупатель=2 отдел=1 окончание=8

[t=4] Выход: покупатель=1 план=выполнен

[t=8] Конец: покупатель=2 отдел=1

[t=8] Выход: покупатель=2 план=выполнен

[t=8] Итог: причина=завершено

В этом сценарии третий покупатель не допускается в магазин, поскольку его плановое прибытие происходит после закрытия. Второй покупатель продолжает обслуживание и выходит в момент 8.

Прерывание во время обслуживания

Пример итоговых сообщений при получении SIGINT:

[t=0] Итог: причина=SIGINT

[t=0] Покупатели: всего=3 вошли=3 вышли=0 внутри=3 не_допущены=0 ещё_не_пришли=0

[t=0] Отдел: номер=1 покупок=0 очередь=2 обслуживается=3

[t=0] Отдел: номер=2 покупок=0 очередь=0 обслуживается=0

[t=0] Отдел: номер=3 покупок=0 очередь=0 обслуживается=0

[t=0] Остался: покупатель=1 состояние=очередь отдел=1 выполнено=0 из=3

[t=0] Остался: покупатель=2 состояние=очередь отдел=1 выполнено=0 из=3

[t=0] Остался: покупатель=3 состояние=обслуживание отдел=1 выполнено=0 из=3

В последнем примере третий покупатель обслуживается, два других ожидают. Их маршруты не отмечаются как выполненные.

Таким образом, программа корректно сохраняет состояние модели при прерывании и не считает незавершённые покупки успешными.

11. Ограничения модели

Количество отделов фиксировано и равно трём. Количество покупателей ограничено 100. Маршруты и интервал появления задаются входным файлом.

Модель конечная, отдельного бесконечного режима нет. Прерывание доступно для запущенного моделирования.

Ассортимент, стоимость товаров и оплата не моделируются. Случайность используется только для выбора длительности обслуживания.

Большая реальная задержка увеличивает время демонстрации, поэтому в автоматических тестах она отключается.

В программе используется общий мьютекс для защиты состояния магазина. Это упрощает синхронизацию и предотвращает одновременное изменение общих данных, но ограничивает параллельное выполнение критических участков кода.

Порядок выполнения потоков определяется операционной системой. Поэтому при одновременном прибытии нескольких покупателей последовательность их постановки в очередь может различаться. При этом правила FIFO, маршруты покупателей и ограничения модели сохраняются.

Проверенные сценарии подтвердили корректность многопоточной реализации. Полностью исключить все возможные ошибки межпоточного выполнения только с помощью тестирования невозможно.

12. Использование ИИ при тестировании и оформлении работы

ChatGPT использовался для подготовки тестовых данных и скриптов проверки, анализа результатов запусков и поиска граничных случаев.

ИИ также помог составить отчёт на основе отчёта по ИДЗ № 1: описать многопоточную модель, взаимодействие потоков, интерфейс программы, способы сборки, запуска и завершения, а также результаты тестирования. Дополнительно были получены пояснения по работе с Git, созданию отдельной ветки и подготовке Pull Request.

Запросы:

Тестирование:

Подготовь тестовые данные и скрипт проверки многопоточной программы. Используй тесты из ИДЗ № 1. Проверь FIFO, порядок посещения отделов, параллельное обслуживание, закрытие магазина, максимальное число покупателей, неверные входные данные и корректную работу синхропримитивов. Укажи ожидаемые результаты.

Анализ результатов:

Проанализируй результаты тестирования многопоточной программы. Проверь отсутствие зависаний, нарушение очередей и корректность взаимодействия потоков. Отдельно проверь завершение по SIGINT и SIGTERM, освобождение ресурсов и сохранение итогового журнала.

Подготовка отчёта:

Составь отчёт по ИДЗ № 2 на основе отчёта по ИДЗ № 1, требований задания, исходного кода и фактических результатов тестирования. Сохрани структуру и стиль предыдущего отчёта. Адаптируй описание модели под многопоточную реализацию, добавь взаимодействие потоков и синхропримитивы, команды сборки и запуска, способы завершения и результаты работы. Не добавляй неподтверждённые сведения.

Работа с GitHub через PowerShell:

Дай команды PowerShell для создания отдельной ветки hw2 в репозитории acos и добавления решения в папку hw2. Сохрани исходное решение ИДЗ № 1 без изменений.